

维护、查找、排序、共享之类的问题。

关于 Linux 文件系统的细节, 请参阅[LOVE05]和[BOVE03]。较好的入门教材是[RUBI97]。

[CUST94] 给出了关于 Windows NT 文件系统的很好的综述。[NAGA97] 更详细地讲述了这方面的问题。

BOVE03 Bovet, D., and Cesati, M. *Understanding the Linux Kernel*. Sebastopol, CA: O'Reilly, 2003.

CUST94 Custer, H. *Inside the Windows NT File System*. Redmond, WA: Microsoft Press, 1994.

FOLK98 Folk, M., and Zoellick, B. *File Structures: An Object-Oriented Approach with C++*. Reading, MA: Addison-Wesley, 1998.

GROS86 Grosshans, D. *File Systems: Design and Implementation*. Englewood Cliffs, NJ: Prentice Hall, 1986.

LIVA90 Livadas, P. *File Structures: Theory and Practice*. Englewood Cliffs, NJ: Prentice Hall, 1990.

LOVE05 Love, R. *Linux Kernel Development*. Waltham, MA: Anovell Press, 2005.

NAGA97 Nagar, R. *Windows NT File System Internals*. Sebastopol, CA: O'Reilly, 1997.

RUBI97 Rubini, A. "The Virtual File System in Linux." *Linux Journal*, May 1997.

WIED87 Wiederhold, G. *File Organization for Database Design*. New York: McGrawHill, 1987.

12.13 关键术语、复习题和习题

关键术语

访问方法	域	文件名	关键域
位表	文件	散列文件	路径名
块	文件分配	索引文件	管道
链接文件分配	文件分配表	索引文件分配	记录
连续文件分配	文件目录	索引顺序文件	顺序文件
数据库	文件管理系统	索引节点	工作目录
磁盘分配表			

复习题

- 12.1 域和记录有什么不同?
- 12.2 文件和数据库有什么不同?
- 12.3 什么是文件管理系统?
- 12.4 选择文件组织时的重要原则是什么?
- 12.5 列出并简单定义五种文件组织。
- 12.6 为什么在索引顺序文件中查找一个记录的平均查找时间小于在顺序文件中的平均查找时间?
- 12.7 对目录执行的典型操作有哪些?
- 12.8 路径名和工作目录有什么关系?
- 12.9 可以授予或拒绝的某个特定用户对某个特定文件的访问权限通常有哪些?
- 12.10 列出并简单定义三种组块方法。
- 12.11 列出并简单定义三种文件分配方法。

习题

- 12.1 定义:

B = 块大小

R = 记录大小

P = 块指针大小

F = 组块因子, 即一个块中期望的记录数

对图 12.6 中描述的三种组块方法分别给出关于 F 的公式。

- 12.2 一种避免预分配中的浪费和缺乏邻近性问题的方案是，分配区的大小随着文件的增长而增加。例如，开始时，分区的大小为一块，在以后每次分配时，分区的大小翻倍。考虑一个有 n 条记录的文件，组块因子为 F ，假设一个简单的一级索引用做一个文件分配表。
- 给出文件分配表中入口数的上限（用关于 F 和 n 的函数表示）。
 - 在任何时候，已分配的文件空间中未被使用的空间的最大量是多少？
- 12.3 当数据
- 很少修改并且以随机顺序频繁地访问时；
 - 频繁地修改并且相对频繁地访问文件整体时；
 - 频繁地修改并以随机顺序频繁地访问时；
- 从访问速度、存储空间的使用和易于更新（添加/删除/修改）这几方面考虑，为了达到最大效率，你将选择哪种文件组织？
- 12.4 忽略目录和文件描述符的开销，考虑一个文件系统，其中文件被存储在大小为 8K 的块中。对于下列各种文件大小，计算文件的最后一块对空间浪费的百分比：3 209 字节；24 576 字节；2 328 432 002 字节。
- 12.5 假设一个 UNIX 系统已经在 `/path` 路径下挂载了一个文件系统。那么一个应用程序为了读取 `/path/to/file` 的第一个字节，必须额外访问多少磁盘块？
- 12.6 目录可以当做一种只能通过受限方式访问的“特殊文件”实现，也可以当做普通文件实现。这两种方法分别有哪些优点和缺点？
- 12.7 一些操作系统具有一个树结构的文件系统，但是把树的深度限制到某个比较小的级数上。这种限制对用户有什么影响？它是如何简化文件系统的设计的（如果能简化）？
- 12.8 考虑一个层次文件系统，空闲的磁盘空间保留在一个空闲空间表中。
- 假设指向空闲空间的指针丢失了。该系统可以重新要空闲空间表吗？
 - 给出一种方案，确保即使出现了一次存储器失败，指针也不会丢失。
- 12.9 一个文件系统有 2KB 个磁盘块，它可以通过索引节点三级索引结构访问到 32GB 大小的数据。试问该文件系统需要用多少位作为块指针？
- 12.10 考虑由一个索引节点表示的 UNIX 文件的组织（见图 12.14）。假设有 12 个直接块指针，在每个索引节点中有一个一级、二级和三级间接指针。此外，假设系统块大小和磁盘扇区大小都是 8K。如果磁盘块指针是 32 位，其中 8 位用于标识物理磁盘，24 位用于标识物理块，那么
- 该系统支持的最大文件大小是多少？
 - 该系统支持的最大文件系统分区是多少？
 - 假设内存中除了文件索引节点外没有别的信息，访问在位置 13 423 956 中的字节需要多少次磁盘访问？

第六部分 嵌入式系统

相比部署在嵌入式系统中操作系统的数量,部署在个人计算机和桌面计算机中的操作系统的数量就相形见绌了。按这种方式衡量,嵌入式操作系统构成了操作系统中最重要的一类。嵌入式操作系统有它们自己独特的需求和设计重点,本部分将研究这些内容。

第六部分导读

第 13 章 嵌入式操作系统

第 13 章首先介绍了嵌入式系统,接着审视嵌入式操作系统的关键特性。然后通过两种广泛使用的系统 eCos 和 TinyOS,来介绍两种非常不同的嵌入式操作系统设计方法。

第 13 章 嵌入式操作系统

本章我们分析一种广泛使用的最重要的操作系统：嵌入式操作系统。嵌入式系统的配置环境独特性、对操作系统的要求苛刻和对设计策略的要求都和构建普通的操作系统大大不同。

我们以嵌入式系统的概念总览开始，然后转而探讨嵌入式操作系统的原理。最后，本章介绍两种很截然不同的嵌入式系统的设计方法。

13.1 嵌入式系统

与通用的计算机（如便携式电脑或桌面系统）不同，嵌入式系统的定义涉及一个产品中电子器件和软件的使用。下面给出一个较好的通用的定义^①：

嵌入式系统是为了完成某个特定功能而设计的，或许有附加的机制或其他部分的计算机硬件和软件的结合体。在许多情况下，嵌入式系统是一个更大的系统或产品中的一部分，比如汽车中的防抱死系统。

嵌入式系统的数量远远超过了普通的操作系统，并且应用非常广泛（见表 13.1）。这些系统的需求和限制有着很大的不同，如下所示[GRIM05]：

表 13.1 嵌入式系统范例与市场[NOER05]

市 场	嵌入式设备
汽车	点火系统 引擎控制 刹车系统
消费电子	数字及模拟电视 机顶盒（DVD、VCR、有线电视） 个人数字助理（PDA） 厨房用具（冰箱、烤箱、微波炉） 汽车 玩具/游戏机 电话/手机/寻呼机 相机 全球定位系统（GPS）
工业控制	制造业的机器人以及控制系统 传感器
医疗	输液泵 透析器 假肢装置 心脏检测器
办公自动化	传真机 复印机 打印机 显示器 扫描仪

① Michael Barr, 《Embedded Systems Glossary》。Netrino 技术图书馆, <http://www.netrino.com/Publications/Glossary/index.php>。

- 从小的系统到大的系统,这意味着非常不同的成本限制,也就是对优化和复用的不同需求。
- 从很宽松的到非常严格的需求,以及不同性质的需求的组合,例如关于安全性、可靠性、实时、灵活性等方面。
- 从很短到很长的使用期限。
- 不同的环境条件,比如辐射、振动、湿度等方面。
- 不同的应用特点,从静态到动态装载,从慢到快的速度,从计算密集型到交互密集型任务,或者它们中的不同组合。
- 不同的计算模型,从离散事件系统到包含连续时间的动态系统(通常也称为混合系统)。

嵌入式系统通常与它们所在的环境紧密地联系在一起。由于与环境交互的需要,从而产生了实时限制。这些限制,如响应速度、测量精度、持续时间等,决定了软件操作的时限。如果多个活动必须进行同步管理,这就需要更复杂的实时限制。

基于[KOOP96],图13.1显示了一般定义下的嵌入式系统组织。除了处理器和内存,还有很多元素不同于传统的台式机或笔记本电脑:

- 或许有不同的接口,以便使系统能进行测量、计算或者与外部环境进行交互的其他接口。
- 用户界面可以是简单到像闪烁的灯一样,也可以像实时机器人视觉那样复杂。
- 诊断端口可以诊断系统是否已经处于被控状态——而不单是诊断计算机。
- 特定目的领域的编程(FPGA)、特定的应用(ASIC)甚至非数字硬件都可能使用,以增强性能或安全性。
- 软件通常是固定功能的,并且特定于某个应用。

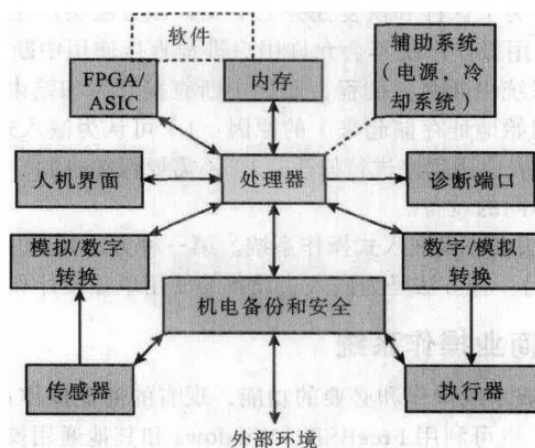


图 13.1 嵌入式系统可能的组织结构

13.2 嵌入式操作系统的特点

一个有着简单功能的嵌入式系统,可以由一个或一组特定的程序,在没有其他软件的情况下进行控制。通常,更复杂的嵌入式系统包括一个操作系统。尽管原则上可以使用一个通用的操作系统,例如 Linux,对于嵌入式系统,存储空间的限制、功耗和实时需求都要求为嵌入式系统环境使用专用的操作系统。

下面列出一些嵌入式操作系统独特的特性和设计需求:

- **实时操作:**在许多嵌入式系统中,计算的准确性部分地依靠交付的时间。通常,实时性受到外围 I/O 和控制稳定性需求的限制。

- **响应操作**：嵌入式软件可以对外部事件响应进行处理。如果这些事件的发生不是周期性的或者是不可预期的，嵌入式软件应该考虑最差情况并为例程的执行设定优先级。
- **可配置性**：由于嵌入式系统的多样性，因此在嵌入式操作系统功能性需求方面是多变的，不论是定量还是定性的。也就是说，一个嵌入式操作系统想要应用在不同的嵌入式系统中时，它自身必须可以灵活配置，以便对特定的应用和硬件系统提供所需的功能。[MARW06]给出了下列例子：链接和加载功能可以用来选择需要加载的 OS 模块。可以使用条件编译。如果使用了一个面向对象结构，就可以定义真子类。实际上，对于许多经过裁剪的衍生操作系统进行设计时，验证是一个潜在的问题。Takada 引用这些作为 eCos 的基本问题[TAKA01]。
- **I/O 设备的灵活性**：事实上，没有设备需要所有版本的操作系统都提供支持，同时 I/O 设备涵盖的范围也很大。[MARW06]中建议，对于处理相应的慢速设备，比如磁盘和网络接口，使用特定任务而不是将这些驱动整合进操作系统的内核中更加合理。
- **改进的保护机制**：嵌入式系统通常为一个受限制的、定义明确的功能而设计。没有经过测试的程序很少加到软件中去。在软件配置和测试之后，就应该被视为可靠的了。也就是说，除安全措施外，嵌入式系统只有有限的保护机制。例如，I/O 指令不必是那些可以陷入操作系统内核的特权指令；任务可以直接完成它们自己的 I/O。类似地，存储保护机制也可以被最小化。[MARW06]提供了下列例子。让 **switch** 对应某个值的内存-映射的 I/O 地址，该值需要作为 I/O 操作的一部分进行检查。我们可以允许 I/O 程序使用一条像 **load register** 这样的指令，**switch** 决定当时的值。这种方法更适合用于一次操作系统服务调用中，为了保存和恢复任务上下文，该方法会产生系统开销。
- **直接使用中断**：通用操作系统不会允许用户进程直接使用中断。[MARW06] 列出了 3 个允许不通过操作系统中断服务例程，而让中断直接开始和结束任务（例如，将中断向量地址表中的任务起始地址存储起来）的原因：1）可认为嵌入式系统是彻底地测试过的，很少对操作系统或者应用程序进行修改；2）不需要保护机制，如前面列出的理由；3）必须能高效地控制不同的设备。

一般有两种通用的方法来开发嵌入式操作系统。第一种方法是利用现有的操作系统，移植它使之适用于嵌入式应用。另一种方法是为嵌入式设备的使用单独设计和实现所需要的操作系统^①。

13.2.1 移植现有的商业操作系统

通过增加实时能力、流水线操作和必要的功能，现有的商业操作系统可以用于嵌入式系统。这种方法通常利用 Linux，也可利用 FreeBSD、Windows 和其他通用操作系统。这些操作系统一般比专用的嵌入式操作系统慢，且可预见性较差。这种方法的优点是直接从一般性商业操作系统派生出嵌入式操作系统，是基于一系列成熟的接口，可以方便地移植。

使用通用操作系统的缺点是它并没有对实时性和嵌入式应用进行优化。也就是说，要达到适当的性能，需要进行很大的修改。特别地，典型的操作系统在调度上是为一般情况进行优化而不是为最差情况进行优化，通常按照需求分配资源，并且忽略大多数关于应用程序的语义信息。

13.2.2 为特定目的构建的嵌入式操作系统

现在已经有数目众多的操作系统是专为嵌入式应用而设计的。后面有两个著名的实例，即 eCos 和 TinyOS，都会在本章中进行讨论。

① 13.2 节的许多讨论是基于位于圣迭戈的加州大学的 Rajesh Gupta 教授提供的嵌入式系统的课堂笔记。网址为 <http://mesl.ucsd.edu/gupta/cse237bw07.html>。

特定的嵌入式操作系统包括如下典型的特点：

- 有快速且轻量级的进程或线程切换。
- 调度策略是实时的，调度模块是调度程序的一部分而不是单独的模块。
- 尺寸很小。
- 能快速响应外部中断，典型的需求是响应时间小于 10 μ s。
- 禁止中断的时间间隔尽可能的小。
- 为内存管理提供固定的或者可变的分区，以及为存储器中的代码和数据加锁的能力。
- 提供特别的顺序文件以便快速存储数据。

为了处理时序约束，内核需要

- 为大部分原语提供有限制的执行时间。
- 维护一个实时的时钟。
- 提供特定的警报和超时。
- 支持实时排队策略，比如最早截止时间优先和将消息插入到队列头的原语。
- 提供固定时间延迟处理的原语和挂起/恢复执行的原语。

上述列出的特性是在实时需求下嵌入式操作系统很常见的特性。实际上，对于复杂的嵌入式系统，需求可能强调可预见性操作胜过快速操作，这就迫使做出不同的设计决策，特别是任务调度方面。

13.3 eCos

在嵌入式应用中，eCos (embedded Configurable operating system) 是一个开源的、版权自由的实时操作系统。该系统的设计目标是高性能的、小型的嵌入式系统。对于这类系统，Linux 或者其他商业操作系统的嵌入式版本不会提供所需要的改进过的软件。eCos 已经广泛应用于不同的处理器平台，包括 Intel IA32、PowerPC、SPARC、ARM、CalmRISC、MIPS 和 NEC V8xx。它是应用最广泛的嵌入式操作系统之一。

13.3.1 可配置性

要想广泛应用于不同的嵌入式应用和嵌入式平台，嵌入式操作系统要足够灵活，必须提供比特定的应用和平台更多的功能。例如，许多实时操作系统支持任务切换、并发控制和不同的优先级调度机制。而相对简单嵌入式系统不一定需要所有这些特性。

为配置可选组件和在组件中允许或禁止特定功能而提供一个高效的、用户友好的机制是一项挑战。可在 Windows 或者 Linux 下运行的 eCos 的配置工具，用于配置运行在目标嵌入式系统上的一个 eCos 包。完整的 eCos 包是分层结构的，使得使用配置工具装配目标配置很容易。在顶层，eCos 由一些组件组成，进行配置工作的用户可以只选择那些目标应用中所需的组件。例如，系统可能有一个特定串口 I/O 设备，用户可以为这个配置选择串口 I/O，然后可以选择一个或者更多特定的受支持的 I/O 设备。配置工具包括为该种支持所需的最小的软件包。配置用户可以选择特定的参数，比如默认的数据速率和要使用的 I/O 缓冲区的大小。

配置过程可以扩展到更加细节的层次，甚至是单独一行代码的层次。例如，配置工具提供包含或者忽略优先级继承协议的选项。

图 13.2 显示了 eCos 配置工具用户可以看到顶层。左手边窗口列表中的每个选项可以选择或不选择。

当一个选项标记为高亮时，右下方的窗口提供了该选项的说明，右上方的窗口提供了进一步说明文档的连接，还有该高亮选项额外的信息。列表中的选项可以展开为更细粒度的选项菜单。

图 13.3 说明了 eCos 内核选项的展开菜单。在图中, 注意异常处理已被选择, 但 SMP (对称多处理) 被忽略。一般来说, 可以选定或者忽略组件或者单个选项。在一些情况下, 可以设定单独的变量值, 例如, 最小的可接受的栈大小是一个整数值, 可以设定该值或者保留默认值。

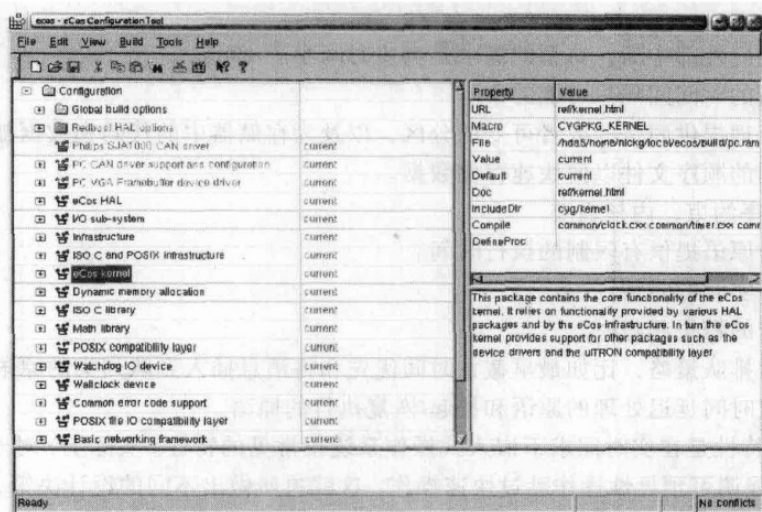


图 13.2 eCos 的配置工具——顶层

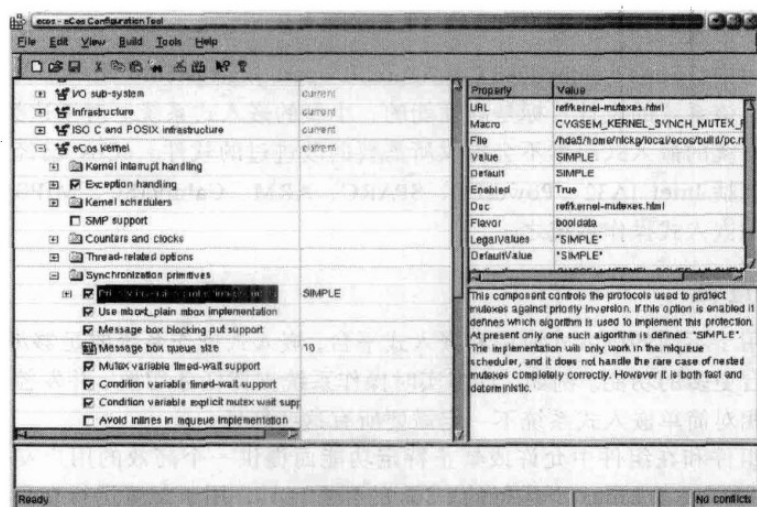


图 13.3 eCos 的配置工具——内核细节

图 13.4 展示了一个典型的例子, 在嵌入式系统中创建可执行二进制映像的整个过程。这个过程运行在宿主机系统上, 比如 Windows 或 Linux 平台, 可执行映像将会在目标嵌入式系统中执行, 比如某个工业环境中的一个传感器。特定的嵌入式应用的应用源码处在最高的软件层。这些源码独立于 eCos, 但是应用编程接口 (API) 处在 eCos 软件的顶层。或者只有一种版本的应用源码, 或者对不同版本的目标嵌入式平台有不同的变种。在这个例子中, GNU make 工具用于选定哪些程序需要编译或者重编译 (在源码修改版本的情况下), 并且发出命令来编译这些源码。GNU 交叉编译器在宿主机上执行, 然后为目标嵌入式平台生成可执行二进制码。GNU 连接器连接应用目标代码和由 eCos 配置工具产生的代码。这个软件集包括选定部分的 eCos 内核加上为目标嵌入式系统选定的软件。其结果随后就可以装入目标系统。

13.3.2 eCos 组件

eCos 的关键设计需求是可移植性，从而对不同的体系结构和平台只需最小的工作量。要满足这个需求，eCos 由一组分层的组件构成（见图 13.5）。

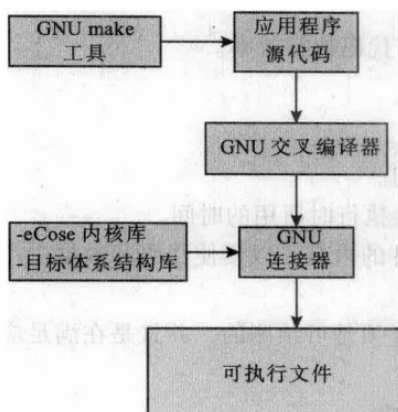


图 13.4 装入一个 eCos 配置

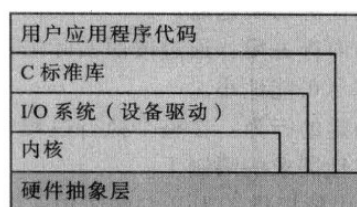


图 13.5 eCos 分层结构

硬件抽象层

最底部是硬件抽象层（HAL）。HAL 是一种软件，它对上层提供一致的 API，并将上层的操作映射到特定的硬件平台，因此每种硬件平台的 HAL 都是不同的。图 13.6 中的例子说明了在两个不同的平台上，为了实现相同的 API 调用，HAL 是怎样进行特定于硬件实现的抽象的。如本例所示，上层使能中断的调用在两个平台上是一样的，但是该功能的 C 代码实现是特定于每个平台的。

```

1 #define HAL_ENABLE_INTERRUPTS() \
2   asm volatile ( \
3     "mrs r3, cpsr;" \
4     "bic r3, r3, #0xc0;" \
5     "mrs cpsr, r3;" \
6     : \
7     : \
8     : "r3" \
9     );
  
```

a) ARM 体系结构

```

1 #define HAL_ENABLE_INTERRUPTS() \
2   CYG_MACRO_START \
3   cyg_uint32 tmp1, tmp2 \
4   asm volatile ( \
5     "mfmsr %0;" \
6     "ori %1,%1,0x800;" \
7     "rlwimi %0,%1,%0,16,16;" \
8     "mtmsr %0;" \
9     : "=r" (tmp1), "=r" (tmp2); \
10  CYG_MACRO_END
  
```

b) PowerPC 体系结构

图 13.6 Hal_Enable_Interrupts()宏的两种实现

HAL 由三个单独的模块实现：

- **体系结构**：定义处理器家族类型。该模块包含支持处理器启动、中断分发、上下文切换、和其他特定于该处理器家族指令集体系结构的特定功能的所需代码。

- **变体**：支持家族中特定处理器的特性。所支持特性的一个例子就是片上模块，比如内存管理单元（MMU）。
- **平台**：将 HAL 扩展以便支持紧密连接的外围设备，如中断控制器和定时器。这个模块定义包括所选择的处理器体系结构和变体的平台或者主板。它包括启动代码、片上配置代码、中断控制器代码和定时器代码。

注意，HAL 的接口可以直接被上层调用，以提高代码执行效率。

eCos 内核

eCos 内核设计主要满足下列四个目标：

- **低中断延迟**：响应中断和开始执行 ISR 的时间。
- **低任务切换延迟**：当一个线程可用并实际开始执行时所用的时间。
- **小内存占用**：通过按照实际需求配置所有组件的内存，以便使代码和数据占用的内存资源尽可能地小。
- **确定的行为**：在整个执行过程中，内核执行必须是可预测的，并且是在满足应用程序实时性需求的限制下。

eCos 内核提供了为开发多线程应用所需的核心功能：

- 1) 在系统中创建新线程的能力，不论是在启动时刻还是在系统的运行时刻。
- 2) 在系统中控制不同的线程，例如，操作它们的优先级。
- 3) 调度器的选择，从而决定哪个线程应该运行。
- 4) 同步原语的范围，允许线程安全地交互和共享数据。
- 5) 系统对中断和异常的支持。

在 eCos 内核中并没有包括操作系统内核的一些典型功能。例如，内存分配由一个独立的包处理。同样，不同的设备驱动也是独立的包。eCos 配置技术将不同的包结合并进行配置以满足应用的需求。这样可以使内核精简。更进一步，对于一些嵌入式平台，最小的内核意味着并没有采用 eCos 内核。简单的单线程应用可以直接在 HAL 上运行。这样的配置可以并入所需的 C 功能库以及设备驱动，但是避免了内核在空间和时间上的开销。

I/O 系统

eCos 的 I/O 系统是一个支持设备驱动的框架。eCos 配置包为大量的平台提供各种驱动。这些驱动包括了串口设备、以太网设备、闪存接口和不同的 I/O 接口如 PCI（外围部件接口）和 USB（统一串行总线）。另外，用户可以开发自己的设备驱动。

I/O 系统的主要目标是效率，舍弃不需要的软件层或者无关功能。设备驱动为输入、输出、缓冲和设备控制提供必要的功能。

如上文所述，如果需要设备驱动和其他更高层的软件，可以直接在 HAL 层上实现。如果需要特定内核类型的功能，设备驱动可以使用内核 API 实现。内核提供了三层中断模型[ECOS07]：

- **中断服务例程（ISR）**：在响应硬件中断时调用。硬件中断在将干扰降到最小的情况下递送至 ISR。HAL 对中断硬件源进行解码，并且调用关联该中断对象的 ISR。这个 ISR 可以操作硬件，但是仅仅允许在设备 API 上进行一组有限制的调用。当返回时，ISR 就会查询它的 DSR 是否应该被调度执行。
- **延迟服务例程（DSR）**：在响应 ISR 的请求时调用。当一个 DSR 不会影响调度时，它是安全的，从而可以运行。大部分时间，DSR 会在 ISR 之后立刻运行，但是当前线程在调度队列中时，在线程结束之前，DSR 会被延迟。DSR 可以调用更多的驱动 API（相对于 ISR），特别是可以调用 `cyg_drv_cond_signal()` 来唤醒等待的线程。

● 线程：驱动的客户。线程可以调用全部的 API，而且可以在互斥量和条件变量上等待。

表 13.2 和 13.3 显示了内核设备驱动接口。这些表对内核支持的设备驱动的可用功能给出了很好的解释。注意，可以为设备驱动接口配置下列一个或多个并发机制：自旋锁、条件变量、互斥量。后面将详细讨论。

表 13.2 eCos 内核设备驱动接口：并发

<code>cyg_drv_spinlock_init</code>	将自旋锁初始化为加锁或者未加锁的状态
<code>cyg_drv_spinlock_destroy</code>	释放一个长时间没有使用的自旋锁
<code>cyg_drv_spinlock_spin</code>	申请一个自旋锁，在忙状态循环中等待，直至该自旋锁可用
<code>cyg_drv_spinlock_clear</code>	清除一个自旋锁。这个操作清除自旋锁，允许其他 CPU 申请它。如果有多于一个 CPU 在 <code>cyg_drv_spinlock_spin</code> 中等待，那么接下来仅有一个 CPU 允许执行
<code>cyg_drv_spinlock_test</code>	检查自旋锁的状态。如果自旋锁没有锁，则返回 TRUE。如果已被锁，则返回 FALSE
<code>cyg_drv_spinlock_spin_intsave</code>	这个功能行为确切说像 <code>cyg_drv_spinlock_spin</code> ，只是它在试图申请自旋锁之前还能禁止中断。当时的中断使能状态保存在 <code>*istate</code> 中。中断在自旋锁申请成功之前都保持禁止中断，并且必须调用 <code>cyg_drv_spinlock_clear_intsave</code> 恢复其状态
<code>cyg_drv_mutex_init</code>	初始化互斥量
<code>cyg_drv_mutex_destroy</code>	销毁互斥量
<code>cyg_drv_mutex_lock</code>	试图为 <code>mutex</code> 参数指向的互斥量加锁。如果互斥量已经被其他线程加锁，则本线程一直等待，直到加锁互斥量的线程结束。如果函数返回值为 FALSE，则本线程解除由其他线程造成的加锁状态。在这种情况下，互斥量不会被锁上
<code>cyg_drv_mutex_trylock</code>	试图为 <code>mutex</code> 参数指向的互斥量加锁，但是不等待。如果互斥量已经被其他线程加锁，则函数返回 FALSE。如果函数可以在不等待的情况下加锁互斥量，则返回 TRUE
<code>cyg_drv_mutex_unlock</code>	为 <code>mutex</code> 参数指向的互斥量解锁。如果已经有一些线程等待请求该锁，则唤醒其中一个从而尝试获取互斥量
<code>cyg_drv_mutex_release</code>	释放所有在该互斥量上等待的线程
<code>cyg_drv_cond_init</code>	初始化一个和某个互斥量关联的条件变量。一个线程在为相关的互斥量加锁后有可能在该条件变量上等待。等待会导致该互斥量的解锁，在该线程再次被唤醒之后，它在继续执行之前会自动申请该互斥量
<code>cyg_drv_cond_destroy</code>	销毁条件变量
<code>cyg_drv_cond_wait</code>	等待条件变量的信号
<code>cyg_drv_cond_signal</code>	向条件变量发送信号。如果有线程在等待这个变量，则至少其中有一个线程将被唤醒
<code>cyg_drv_cond_broadcast</code>	向条件变量发送信号。如果有线程在等待这个变量，则所有线程都会被唤醒

表 13.3 eCos 内核设备驱动接口：中断

<code>cyg_drv_isr_lock</code>	禁止中断的分发，阻止所有 ISR 的运行。本函数维护着一个它被调用的次数的计数器
<code>cyg_drv_isr_unlock</code>	使能中断的分发，允许 ISR 运行。本函数递减由 <code>cyg_drv_isr_lock</code> 维护的计数器，当计数器为 0 时，再次允许中断
<code>cyg_isr_t</code>	定义 ISR

(续)

<code>cyg_drv_dsr_lock</code>	禁止 DSR 的调度。本函数维护着一个它被调用的次数的计数器
<code>cyg_drv_dsr_unlock</code>	使能调度 DSR。本函数递减由 <code>cyg_drv_dsr_lock</code> 维护的计数器。只有当计数器变为 0 时, 才会允许 DSR 的分发
<code>cyg_DSR_t</code>	定义 DSR 原型
<code>cyg_drv_interrupt_create</code>	创建一个中断对象并且返回一个处理句柄
<code>cyg_drv_interrupt_delete</code>	从中断向量表中删除中断, 并且释放内存以便再次使用
<code>cyg_drv_interrupt_attach</code>	向向量表中添加中断, 当中断发生时, 中断就可以传递到 ISR
<code>cyg_drv_interrupt_detach</code>	从中断向量表中删除中断, 从而该中断不再被分发给 ISR
<code>cyg_drv_interrupt_mask</code>	对中断控制器进行编程, 从而在给定的向量表中停止中断的分发
<code>cyg_drv_interrupt_mask_intunsafe</code>	对中断控制器进行编程, 从而在给定的向量表中停止中断的分发。此版本与 <code>cyg_drv_interrupt_mask</code> 的不同之处在于其中断是不安全的。因此在诸如已知中断被禁止的情况下, 这样可以避免额外的开销
<code>cyg_drv_interrupt_unmask, cyg_drv_interrupt_unmask_intunsafe</code>	对中断控制器编程, 使得可以在给定的向量表中再次允许分发中断
<code>cyg_drv_interrupt_acknowledge</code>	完成中断控制器中所需任意的处理, 并且在 CPU 中取消当前中断请求
<code>cyg_drv_interrupt_configure</code>	按照中断源的特征进行中断控制器的编程
<code>cyg_drv_interrupt_level</code>	对中断控制器进行编程, 从而按照提供的优先级级别分发中断
<code>cyg_drv_interrupt_set_cpu</code>	在多处理器系统中, 此函数使在给定向量表中的所有中断可以发送到特定的 CPU。随后, 该 CPU 可以处理相应的中断
<code>cyg_drv_interrupt_get_cpu</code>	在多处理器系统中, 该函数返回当前正在分发的中断的目标 CPU 的 ID

C 标准库

提供完整的 C 运行时标准库。同时包括一个用于高层次数学函数的完整的数学运行时库, 以及一个用于没有硬件浮点功能的平台的完整的 IEEE-754 浮点库。

13.3.3 eCos 调度程序

eCos 的内核可以在已设计好的两种调度程序中选择一种进行配置: 位图调度和多级队列调度。进行配置的用户针对其环境和应用选择合适的调度程序。位图调度适用于在任何时间点上活动的线程数目较少的系统, 位图调度可以为其提供高效的调度。多级队列调度适用于线程数量是动态的或者有多个相同优先级的线程的情况。多级队列调度也同样适用于时间片的情况。

位图调度

位图调度支持多优先级, 但是在任意时刻, 每个优先级只能有一个线程存在。在此调度程序中, 调度决策是相当简单的 (见图 13.7a)。当一个阻塞的线程变为准备运行时, 它可以抢占优先级更低的线程。当一个线程挂起时, 调度高优先级处于准备状态的线程。一个线程可能由于在同步原语上阻塞、被中断或是放弃控制而被挂起。因为在每个优先级上至多只有一个线程, 所以调度程序无须决定在给定的优先级中下次待调度的线程。

位图调度可配置为 8、16 或 32 个优先级。为准备执行的线程准备一个简单的位图。为了做出调度决定, 调度程序只需要确定该位图的最高为 1 的位的位置即可做出调度决策。

多级队列调度

同位图调度一样, 多级队列调度也支持 32 个优先级。多级队列调度在每个优先级上允许有多个活动的线程, 其线程数目仅受系统资源限制。

图 13.7b 描述了多级队列调度。图中的数据结构代表了每个优先级中线程的数目。当一个阻塞的线程变为准备运行状态时, 它可以抢占更低优先级的线程。同位图调度一样, 一个正在运行的线程可能由于在同步原语上阻塞、中断或是放弃控制而被挂起。当一个线程被阻塞, 调度程序

首先确定是否有一个或多个与该阻塞线程有相同优先级的线程处于就绪状态,如果确实如此,调度程序选择该队列最前面的一个进行调度。否则,调度程序寻找比当前优先级低的最高优先级中的一个或多个准备状态的线程并且派送一个线程。

另外,多级队列调度可以配置为时间片调度。这样的话,如果一个线程正在运行,并且同时有一个或多个准备状态的相同优先级的线程,调度程序将在一个时间片结束后挂起当前运行进程,并且选择同优先级队列中的下一个线程。这个轮转策略只在同一优先级中进行。并非所有的应用程序都需要时间片调度。例如,一个应用程序可以保存只是因为相同原因而有规律地被阻塞的线程。对于这些应用程序,用户可以禁止时间片,这会减小由于定时器中断导致的开销。

13.3.4 eCos 线程同步

可以为 eCos 内核配置六种线程同步机制中的一种或多种。这些同步机制包括经典的同步机制:互斥量、信号量和条件变量。此外, eCos 支持两个同步/通信机制,这在实时系统中很常见,也就是事件标志和信箱。最后, eCos 内核支持自旋锁,这在 SMP (对称多处理机) 系统中很有用。

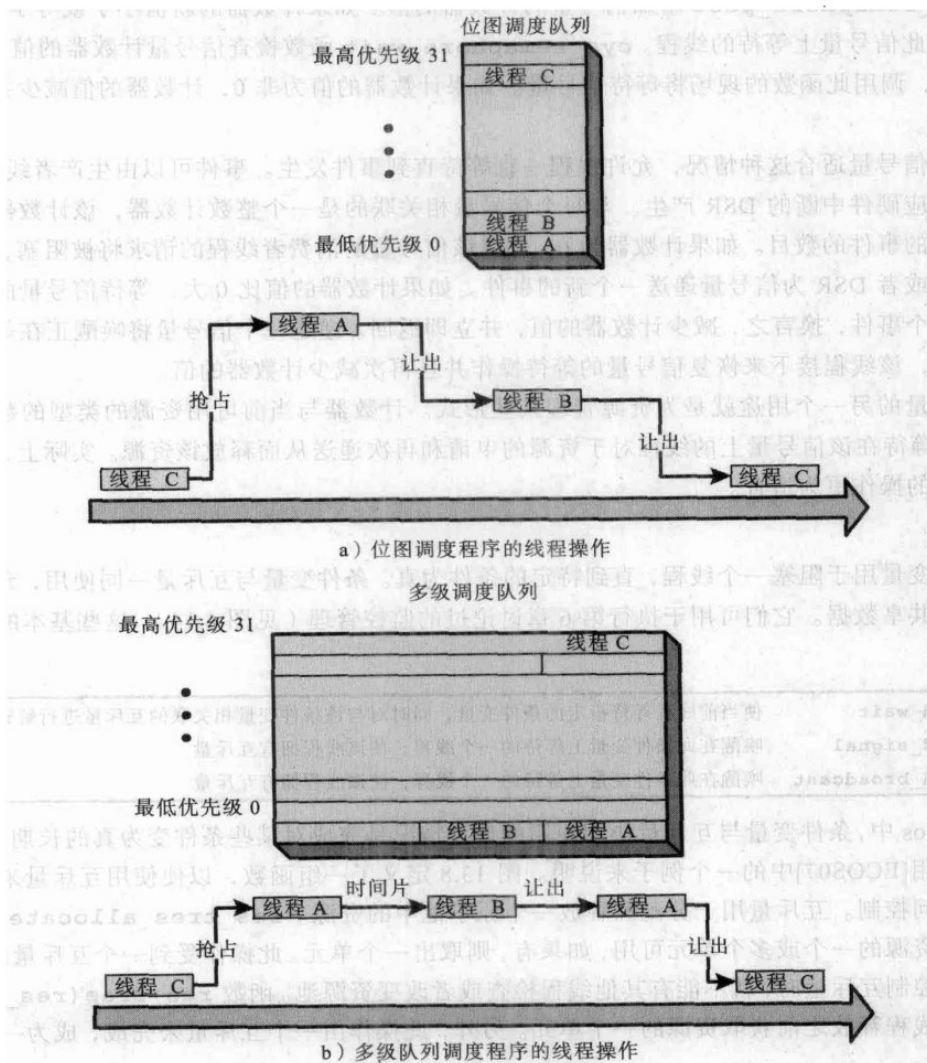


图 13.7 eCos 调度操作

互斥量

互斥量（互斥锁）已经在第 6 章介绍过了。回想一下，互斥量用于强制互斥访问资源，在同一时间只有一个线程允许获得访问权。互斥量只有两种状态：加锁和解锁。这与二元信号量有些类似：当一个线程锁住一个互斥量后，其他试图锁住互斥量的线程被阻塞；当互斥量解锁后，在此互斥量上被阻塞线程的其中之一解除阻塞，并且允许加锁互斥量以及获取资源访问权。

互斥量在两方面不同于二元信号量。首先，给互斥量加锁的线程必须给互斥量解锁。相比之下，二元信号量允许一个线程加锁而另一个线程为其解锁。另一处不同之处是信号量对优先级反转提供保护，而信号量不提供保护。

ECos 内核可以配置为既支持优先级继承协议又支持优先级置顶协议。这些在第 10 章中已经介绍过。

信号量

eCos 内核支持计数信号量。回忆下第 5 章，计数信号量是一个整数值，用于为线程发送信号。`cyg_semaphore_post` 增加信号量的计数器的值。如果计数器的新值小于或等于 0，则唤醒一个在此信号量上等待的线程。`cyg_semaphore_wait` 函数检查信号量计数器的值。如果计数器为 0，调用此函数的现场将等待信号量。如果计数器的值为非 0，计数器的值减少并且线程继续执行。

计数信号量适合这种情况，允许线程一直等待直到事件发生。事件可以由生产者线程产生，或者由响应硬件中断的 DSR 产生。与每个信号量相关联的是一个整数计数器，该计数器保存着尚未处理的事件的数目。如果计数器为 0，等待该信号量的消费者线程的请求将被阻塞，直到其他的线程或者 DSR 为信号量递送一个新的事件。如果计数器的值比 0 大，等待信号量的请求就会消费一个事件，换言之，减少计数器的值，并立即返回。递送一个信号量将唤醒正在等待的第一个线程，该线程接下来恢复信号量的等待操作并且再次减少计数器的值。

信号量的另一个用途就是为资源管理某些形式。计数器与当前可用资源的类型的数目相符合，以及等待在该信号量上的线程对于资源的申请和再次递送从而释放该资源。实际上，条件变量对类似的操作更为适合。

条件变量

条件变量用于阻塞一个线程，直到特定的条件为真。条件变量与互斥量一同使用，允许多个线程访问共享数据。它们可用于执行第 6 章讨论过的监控管理（见图 6.14）。这些基本的命令如下：

<code>cyg_cond_wait</code>	使当前线程等待指定的条件变量，同时对与该条件变量相关联的互斥量进行解锁
<code>cyg_cond_signal</code>	唤醒在此条件变量上等待的一个线程，使该线程拥有互斥量
<code>cyg_cond_broadcast</code>	唤醒在此条件变量上等待的一个线程，使该线程拥有互斥量

在 eCos 中，条件变量与互斥量协同使用的典型情况是完成对某些条件变为真的长期的等待。我们可以用[ECOS07]中的一个例子来说明。图 13.8 定义了一组函数，以便使用互斥量来对资源池进行访问控制。互斥量用于分配和释放一个原子池中的资源。`res_tres_allocate` 函数检查是否有资源的一个或多个单元可用，如果有，则取出一个单元。此操作受到一个互斥量的保护，当此线程控制互斥量时，就不能有其他线程检查或者改变资源池。函数 `res_free(res_tres)` 允许一个线程释放之前获取资源的一个单元。另外，此操作由一个互斥量来完成，成为一个原子操作。

在这个例子中，如果一个线程试图访问一个资源但却没有可用的资源，函数返回 `RES_NONE`。

设想一下，实际上，我们希望线程能被阻塞并且等待，直到一个资源变得可用，而不是返回一个 **RES_NONE**。图 13.9 使用与互斥量相关联的条件变量来完成这个操作。当 **res_allocate** 检测到没有可用的资源，它将调用 **cyq_cond_wait**。后者对互斥量进行解锁，并且调用在条件变量上睡眠的线程。当 **res_free** 最终被调用时，它将一个资源放回资源池，并且调用 **cyq_cond_signal** 来唤醒任意一个等待在条件变量上的线程。当等待的线程最终再次运行时，它将在 **cyq_cond_wait** 返回之前再次加锁互斥量。

```

Cyg_mutex_t res_lock;
res_t res_pool[RES_MAX];
int res_count = RES_MAX;

void res_init(void)
{
    cyg_mutex_init(&res_lock);
    <fill pool with resources>
}

res_t res_allocate(void)
{
    res_t res;

    cyg_mutex_lock(&res_lock); // 给互斥量加锁

    if( res_count == 0 )        // 检查空闲资源
        res = RES_NONE;        // 如果没有空闲资源返回 RES_NONE
    else
    {
        res_count--;           // 分配资源
        res = res_pool[res_count];
    }

    cyg_mutex_unlock(&res_lock); // 对互斥量解锁

    return res;
}

void res_free(res_t res)
{
    cyg_mutex_lock(&res_lock); // 给互斥量加锁

    res_pool[res_count] = res; // 释放资源
    res_count++;

    cyg_mutex_unlock(&res_lock); // 对互斥量解锁
}

```

图 13.8 使用互斥量对资源池进行访问控制

[ECOS07]指出了比较重要的两点，以及一般情况下条件变量的使用。首先，互斥量解锁和等待 **cyq_cond_wait** 是原子操作：在解锁到等待期间没有其他线程可以运行。如果不是原子操作的话，由其他线程调用的 **res_free** 将会释放资源，但是这样就会丢失对 **cyq_cond_signal** 的调用，当有资源可用时，第一个线程就会结束等待。

第二个特性是，对 **cyq_cond_wait** 的调用是在一个 **while** 循环之内的，而不是一个简单的 **if** 声明。这是因为当收到信号的线程被再次唤醒时，需要在 **cyq_cond_wait** 中再次对互斥量进行加锁。依靠调度程序和队列顺序，许多线程可以在该线程运行之前进入临界区。因此它等待的条件就可能已经递送了 **false**。在所有的等待操作的条件变量中使用循环是保证条件在等待之后仍然保持 **true** 的唯一方法。

事件标志

事件标志是一个 32 位的字，作为一个同步机制来使用。应用程序的代码可以为每一位关联

一个事件。通过查询相应的一个或多个标志位,线程可以等待一个单独的事件或多个关联的事件。在所有需要的位都被置位 (AND) 或者有一个位被置位 (OR) 之前,线程处于阻塞状态。根据特定的条件或事件,一个发信号的线程可以设定或者再设置位,从而相应的线程被阻塞。例如,第 0 位可以表示一个特定的 I/O 操作完成,使数据可用,第 1 位可以表示用户按了启动键。一个生产者线程或 DSR 可以将这两个位置位,而一直等待这两个事件的消费者线程被唤醒。

```

cyg_mutex_t res_lock;
cyg_cond_t res_wait;
res_t res_pool[RES_MAX];
int res_count = RES_MAX;

void res_init(void)
{
    cyg_mutex_init(&res_lock);
    cyg_cond_init(&res_wait, &res_lock);
    <fill pool with resources>
}

res_t res_allocate(void)
{
    res_t res;

    cyg_mutex_lock(&res_lock);    // 对互斥量加锁

    while( res_count == 0 )        // 等待一个资源
        cyg_cond_wait(&res_wait);

    res_count--;                  // 分配资源
    res = res_pool[res_count];

    cyg_mutex_unlock(&res_lock);  // 对互斥量加锁

    return res;
}

void res_free(res_t res)
{
    cyg_mutex_lock(&res_lock);    // 对互斥量加锁

    res_pool[res_count] = res;    // 释放资源
    res_count++;

    cyg_cond_signal(&res_wait);  // 唤醒任何等待的分配符

    cyg_mutex_unlock(&res_lock); // 对互斥量解锁
}

```

图 13.9 使用互斥量和条件变量对资源池进行访问控制

通过使用 **cyg_flag_wait**, 一个线程可以等待一个或多个事件, 该函数有三个参数: 一个特定事件的标志, 标识中相关联的位的位置组合以及一个模式参数。模式参数指定了在所有位被置位 (AND) 或者至少一个位被置位 (OR) 之前, 线程是否被阻塞。模式参数也可以指定什么时候等待完成, 全部事件标识被清空 (全部设定为 0)。

信箱

信箱, 当然也称为邮箱, 是一种 eCos 的同步机制, 它提供两个线程交换信息的方法。5.5 节进行了一个关于消息传递同步的一般性讨论。在这里, 我们看看 eCos 版本下的细节。

eCos 信箱机制在发送端或者接收端都可以配置为阻塞或非阻塞状态。对一个给定的信箱,

其消息队列大小的最大值也可以进行配置。

发送信息的原语, 称为 `put`, 包括两个参数: 一个信箱的句柄以及一个指向消息本身的指针。这个原语有三个变量:

<code>cyg_mbox_put</code>	如果信箱中有一个空闲的槽, 新消息就可以放在该槽中; 如果有一个等待的线程, 将唤醒它, 这样该线程就可以接收消息。如果信箱正好满了, <code>cyg_mbox_put</code> 就会阻塞, 直到有一个相应的 <code>get</code> 操作使一个槽可用为止
<code>cyg_mbox_timed_put</code>	如果有一个空闲的槽, 则类似于 <code>cyg_mbox_put</code> 。否则, 函数会在一个指定的时间期限中等待, 如果这期间有一个槽可用, 则发送消息。如果超过该时间期限, 操作返回 <code>false</code> 。也就是说, <code>cyg_mbox_timed_put</code> 阻塞的时间小于或等于一个指定的时间间隔
<code>cyg_mbox_tryput</code>	这是一个非阻塞的版本, 当消息发送成功返回 <code>true</code> , 当信箱满时返回 <code>false</code>

同样的, 对于 `get` 原语, 也有三个参数:

<code>cyg_mbox_get</code>	如果在指定的信箱中有一条未决的消息, <code>cyg_mbox_get</code> 返回已经放入信箱的该消息。否则, 函数将一直阻塞, 直到有一个 <code>put</code> 操作
<code>cyg_mbox_timed_get</code>	如果一条消息可用则立刻返回。否则, 函数将一直等待, 直到一条消息可用或者直到很多时钟周期过去。如果超时, 操作返回一个空指针。也就是说, <code>cyg_mbox_timed_get</code> 阻塞的时间小于或等于指定的时间间隔
<code>cyg_mbox_tryget</code>	这是一个非阻塞的版本, 如果消息可用它返回一条消息, 如果信箱为空, 则返回空指针

自旋锁

自旋锁是一个标志位, 一个线程在执行一段特定的代码之前可以对其进行检测。回忆一下第 6 章对 Linux 自旋锁的讨论, 自旋锁基本的操作是: 同一时间只有一个线程能够获取一个自旋锁。任何其他的线程试图获取该自旋锁将被保持尝试状态 (自旋), 直到能够获取该自旋锁为止。实质上, 自旋锁就是建立在内存区的一个整数上, 任何进程进入临界区之前都要检查该整数。如果值为 0, 线程将设置该值为 1, 并且进入临界区。如果值为非 0, 线程就会一直检查该值, 直到它为 0 为止。

自旋锁不能用于单处理机系统, 这就是 Linux 为什么将它单独编译的原因。作为一个有风险的例子, 思考一下在可抢占调度下的单处理机系统, 一个高优先级的线程试图获取一个低优先级已经掌控的自旋锁。因为高优先级的线程抢占了低优先级的线程, 低优先级的线程不能执行, 从而结束它的工作并释放自旋锁。高优先级的线程可以执行但却一直陷在检查自旋锁中。在 SMP 系统中, 自旋锁当前的用户可以继续在不同的处理机上运行。

13.4 TinyOS

与一般商业性通用操作系统相比, 如嵌入式 Linux, eCos 为嵌入式操作系统提供了更新式的方法。因而, 对内存、处理时间、实时响应、功耗等要求较严格的情况下, eCos 和类似的系统更适合此类情况。TinyOS 系统非常精简, 为嵌入式系统提供了最小的操作系统。它的核心操作系统的代码和数据仅需要 400 个字节的内存。

TinyOS 与其他嵌入式操作系统有着显著的不同。一个明显的不同之处是 TinyOS 不是实时操作系统。这个是因为预期的工作负载的原因, 该工作负载主要产生在无线传感器网络上下文, 下一小节将详细描述。因为功耗的原因, 这些设备大部分时间是关闭的。应用程序趋向于简单, 处理器的抢占也不再是问题的重点。

另外, TinyOS 没有内核, 这是因为它是没有存储保护, 是基于组件的操作系统; 系统中没有进程; 操作系统本身也没有存储分配系统 (尽管有些很少用到的组件引进一个存储分配系统); 中断和异常处理依靠外围设备; 并且它是完全无阻塞的, 因此很少有直接同步原语。

TinyOS 已经成为一个实现无线传感器网络软件的流行方法。目前, 超过 500 家组织都在为 TinyOS 开发和发布开源标准。

13.4.1 无线传感器网络

TinyOS 最初是为使用较少的无线传感器网络而开发的。一些趋势使得开发极紧凑的、低功耗的传感器成为可能。在著名的摩尔定律驱使下, 存储器和处理机逻辑部件的尺寸一再降低。更小的尺寸减少了功耗。在无线通信硬件、微型机电传感器 (MEMS) 和传感器中, 低功耗和小尺寸的趋势也非常明显。结果是在一立方毫米内, 开发一个包含逻辑电路的完整的传感器变为可能。应用软件和系统软件必须足够紧凑, 传感、通信和计算能力可以组成一个完整但是微小的体系结构。

低成本、小尺寸、低功耗的无线传感器可以用于许多应用中[ROME04]。图 13.10 展示了一个典型的配置。一个基站将传感器网络和 PC 主机连接起来, 并且通过网络将传感器的数据传送到 PC 主机上, 该主机可以进行数据分析和/或通过企业网或因特网向分析服务器传输数据。单个的传感器搜集并传送数据到基站中, 既可以直接传送, 也可以通过数据转播的方式经过其他传感器传送。路由功能是必需的, 用来决定怎样转播数据, 以便通过传感器网络到达基站。[BUON01] 指出, 在许多应用中, 用户希望能够快速布置许多低成本的设备而无需配置或管理它们。这意味着这些设备必须能将自己整合入一个特别的网络中去。个体传感器的移动性和射频 (RF) 接口意味着网络能够在秒的数量级上进行重新自配置。

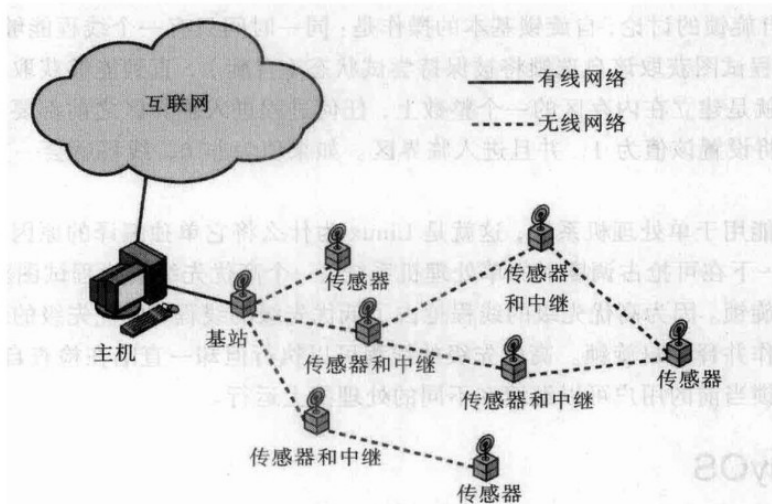


图 13.10 典型的无线传感器网络技术

13.4.2 TinyOS 的目标

考虑到微型的、分布式的传感器应用, 伯克利大学的一个研究者团队[HILL00]为 TinyOS 设定了下列目标:

- **允许高并发性**: 在一个典型的无线传感器网络应用中, 设备要求有强并发性。几个不同的数据流必须保持同步移动。当传感器数据在一个稳定的流中输入时, 处理结果也必须在稳定的流中进行传送。另外, 必须管理对遥控传感器和基站的外部控制。

- 在有限的资源下操作：TinyOS 的目标平台会受到存储器和计算资源的限制，并且靠电池或者太阳能运行。一个单一的平台可能仅提供几 K 字节的程序存储器和几百字节的随机存储器（RAM）。软件必须高效地使用可用的处理器和存储器资源，并且允许低功耗通信。
- 适应硬件升级：硬件始终在发展演化；应用软件和大部分系统服务必须兼容硬件换代。也就是说，如果功能相同的话，应该可以在升级硬件时没有或者只有很少的软件改变。
- 支持广泛的应用软件：应用软件在其生命周期内展示了很广的需求范围，通信、传感等。因此需要模块化、通用的嵌入式操作系统，以便在开发和支持软件时有标准化的方法来节约成本。
- 支持不同的平台：如前所述，需要通用的嵌入式操作系统。
- 是可靠的：传感器网络一旦部署，就必须在无人监护的状态下运行数月或数年。理想情况下，在单一的系统中和全部的传感器网络都应该设计冗余备用系统。实际上，两种类型的冗余都需要额外的资源。能够提高可靠性的软件的特点是使用高模块化、标准化的软件组件。

需要详细叙述并发的需求。在典型的应用中，可能会有几打、几百个甚至几千个传感器连成一个网络。由于延迟时间的原因，很少使用缓冲。例如，如果每隔 5 分钟采样一次，并且希望在发送之前缓存 4 个样本，平均延迟时间就是 10 分钟。这样的话，信息通常被捕获、处理并发送到网络中去，这一切都完成在一个连续的流中。进一步来说，如果传感器采样产生很大的数据，有限的存储器可用空间限制了可被缓存的采样的数目。虽然如此，在一些应用中，每个流包括很多低级别的事件，这些事件与高级别的处理交叉在一起。一些高级别的处理会扩展为多个实时事件。更进一步来说，网络中的传感器，因为传输的低功耗，通常只能在一个短的物理范围内完成操作。从外部传感器传送来的数据必须经过中间节点传递到一个或多个基站。

13.4.3 TinyOS 的组件

使用 TinyOS 构建一个嵌入式系统由一系列的小模块组成，称为组件，每一个组件完成一个简单的任务或者一组任务，每个组件与其他组件和硬件的接口受到一定限制且定义明确。仅有的一个例外的软件模块是调度程序，稍后将进行讨论。实际上，因为没有内核，也就没有实际的操作系统。但是我们可以采用下面的观点。主要的应用领域是无线传感器网络（WSN）。为了满足该应用的软件需求，就需要一个严格的、包含各种组件简化的软件架构。TinyOS 的开发团队已经完成了许多开源组件，这些组件为 WSN 提供了所需的基本函数。这些标准组件的例子包括单跳网络（single-hop networking）、自组织路由（ad-hoc routing）、电源管理、定时器和非易失存储控制。对于特定的配置和应用，用户构建额外的特殊目的的组件，连接并装入用户应用软件的全部组件中。TinyOS 由一系列标准化组件组成。对于任意特定的实现，并非所有的组件都能用得上，何况还有一些用户编写的特定应用的组件。上述实现中的操作系统仅仅是 TinyOS 套件中的标准组件简单的集合。

所有配置在 TinyOS 中的组件有着相同的结构，图 13.11a 展示了一个例子。图中带阴影的方框表示组件，该组件作为一个对象，只能通过定义的接口进行访问，这些接口由白色的方框表示。组件可以是硬件或软件。软件组件由 nesC 实现，nesC 是 C 语言的一个扩展，有两个明显的特征：通过接口与组件进行交互的编程模型和带有从运行到完成任务和中断句柄的基于事件的并发模型，随后将进行讨论。

体系结构由分层排列的组件构成。每个组件仅可以连接其他两个组件，一个比它的层次低，一个比它的层次高。组件向比它低层的组件发出命令并从中接收事件信号。类似地，组件从比它高层的组件中接收命令并且向其发送事件信号。最底层的级别是硬件组件，最顶层的级别是应用软件组件，该组件可能不是 TinyOS 标准套装中的一部分，但必须符合 TinyOS 组件的结构。

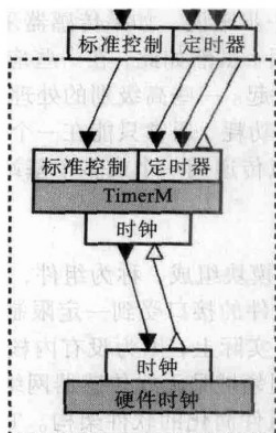
软件组件执行一个或多个任务。每个组件内的任务类似于普通操作系统中的线程，但有一些限制。在一个组件内，任务是原子的：一旦任务开始执行，就要运行到完成。在相同的组件内，不能被别的任务抢占，也没有时间分片。然而，任务可以被事件抢占。任务不能阻塞或自旋等待。这些限制大大简化了组件内的调度和管理。只有一个简单的栈，分配给了当前运行的任务。任务可以完成计算，调用低层次组件（命令），向高层次的事件发送信号，还可以调度其他任务。

命令是不可阻塞的请求。也就是说，任务发送了一条命令，不能在低层次的组件回应阻塞或自旋等待。命令一般是让低层次的组件完成某些服务的请求，比如初始化一个传感器的读操作。对于组件的影响来说，接受命令的组件的效果是特定于给定命令以及运行该命令的任务的。一般情况下，当接收到一条命令，其后的执行就是调度任务，因为命令不可以抢占当前运行的任务。命令立刻返回，调用组件；稍后，事件将对调用组件发送完成信号。也就是说，在被调用的组件中，命令不会导致抢占，在调用的组件中，命令不会导致阻塞。



```
module TimerM {
  provides {
    interface StdControl;
    interface Timer;
  }
  uses interface Clock as Clk;
  ...
}
```

a) TimerM 组件



```
configuration TimerC {
  provides {
    interface StdControl;
    interface Timer;
  }
}

implementation {
  components TimerM, HWClock;
  StdControl = TimerM.StdControl;
  Timer = TimerM.Timer;
  TimerM.Clk -> HWClock.Clock;
}
```

b) TimerC 配置

图 13.11 组件和配置的例子

TinyOS 中的事件可以直接或者间接地与硬件事件相关联。最低层的软件组件接口直接对应硬件中断，中断可以是外部中断、定时器事件或计数器事件。底层组件的事件处理句柄可以自己处理中断或者向组件上层传递事件消息。命令可以传递一个任务，此后会发送一个事件信号。在这种情况下，与硬件事件毫无关联。

任务可以视为有三个阶段。调用者向模块发出一条命令。模块接着响应任务。然后模块通过事件通知调用者，任务已经完成。

图 13.11a 中描述的组件，TimerM 是 TinyOS 定时器服务的一部分。这个组件提供标准控制和定时器接口以及时钟接口。提供者实现命令（组件中的逻辑），用户实现事件（组件外部）。许多 TinyOS 组件使用标准控制接口来进行初始化、启动或停止。TimerM 提供如下逻辑，将硬件时钟映射到 TinyOS 的定时器抽象中。时钟抽象可以为指定的时钟间隔进行倒计时操作。图 13.11a 同样显示了定时器 M 接口的形式规格说明。

与定时器 M 关联的接口定义如下：

```
interface StdControl {
    command result_t init();
    command result_t start();
    command result_t stop();
}
interface Timer {
    command result_t start(char type, uint32_t interval);
    command result_t stop();
    event result_t fired();
}
interface Clock {
    command result_t setRate(char interval, char scale);
    event result_t fire();
}
```

组件通过在它们的接口上“布线”来组织为配置，并且等同于带有组件中一些接口的配置接口。简单的例子见图 13.11b。大写 C 代表组件，这是为了区别接口（如定时器）和提供接口的组件（如定时器 C）。大写 M 代表模块。当一个逻辑组件既有配置又有模块时采用这样的命名规则。定时器 C 组件，提供定时器接口，该接口为时钟和 LED 提供者连接它的执行（TimerM）。另外，任何使用 TimerC 的用户必须明确地与它的子部件相连。

13.4.4 TinyOS 的调度程序

TinyOS 的调度程序操作贯穿整个组件。实际上，所有使用 TinyOS 的嵌入式系统都是单处理机系统，从而在同一时间，全部组件的全部任务中，只有一个任务在执行。调度程序是一个单独的组件。在任何使用 TinyOS 的系统中都必须存在的一部分。

TinyOS 中默认的调度程序是一个简单的 FIFO 队列。任务递送给调度程序（放入队列）或者作为一个事件的结果，这可以触发递送，或者作为一个正在运行的任务调度另一个任务的请求的结果。调度程序是节能的，这就意味着当没有任务在队列里时，调度程序使处理器休眠。外围设备保持操作，通过硬件事件向最底层的组件发送信号，这些设备中的某个可以唤醒系统。一旦队列清空，其他任务就只能作为一个直接硬件事件的结果来被调度。这一行为使得高效的电池利用成为可能。

调度程序经历过两代发展。在 TinyOS 1.x 中，针对所有的任务有一个共享的任务队列，一个组件可以多次递送一个任务到调度程序中。如果任务队列已满，递送操作失败。网络栈的设计经验显示了这样做可能有些问题，任务可以发送分阶段操作完成信号：如果发送失败，上述组件可能永远阻塞，等待一个完成事件。在 TinyOS 2.x 中，每个任务在任务队列中都有自己的保留槽，一个任务只能被发送一次。只有任务已经被发送过才会出现发送失败。如果一个组件需要多次发送一个任务，它可以设定一个间隔状态变量，当任务执行时，它自己进行发送。这个语义上微小的编号简化了许多组件代码。在发送一个任务前，比先测试一下任务是否已经发送过该任务更好的方法是，组件可以发送任务。组件不必尝试从失败和再次尝试中恢复。代价是每个任务有一个字节来表示状态。用户可以使用不同的调度策略来取代默认的调度程序，例如基于优先级调度或者时间期限调度。实际上，不再使用抢占和时间片，因为这类系统产生系统开销。更重要的是，它们违反了 TinyOS 当前的模型，该模型假定任务不许彼此抢占。

13.4.5 配置例子

图 13.12 显示了一个由硬件和软件组件组成的结构。这是个简单的例子，叫做 Surge，在 [GAY03] 中有详细描述，该例子完成周期性传感器采样并且使用自组织多跳路由（ad-hoc multihop routing），以便在无线网络中传递样本至基站。图的上半部分显示了 Surge 的组件（由方框表示）